Mechanical and Aerospace Engineering
Machine Learning in Fluid Flow
Dr. Qiong Liu

Project 1

Luis C. Diaz Dominguez
800772032
luisper2@nmsu.edu
September 30th, 2025

Contents

Problem 1: Singular Value Descomposition

Approach

The Singular Value Decomposition (SVD) is a fundamental matrix factorization technique in linear algebra. For any real matrix $A \in \mathbb{R}^{m \times n}$, the SVD is given by

$$A = U\Sigma V^T,$$

where U and V are orthogonal matrices and Σ is a diagonal matrix containing the singular values of A. These singular values represent the importance of each corresponding mode in the matrix. Beyond its role in linear algebra, the SVD is also widely used in optimization problems, since it provides the best low-rank approximation of a matrix in the least-squares sense. This means that by truncating the smaller singular values, we can compress data, reduce dimensionality, and accelerate computations while still preserving most of the relevant information.

In this problem, we generated a 100×100 random matrix sampled from a normal distribution. The SVD was computed, and the singular values were plotted. This process was repeated 100 times to obtain a statistical distribution of singular values.

Solution

```python
import numpy as np
from tqdm import tqdm
import matplotlib.pylab as plt

# Define the variables
trials = 100 # n Trials
n = 100      # Matrix size n x n
cache = []   # Save the Singular values for each iteration

# Iterate the process to achive the number of trials
for i in tqdm(range(trials)):
    matrix = np.random.randn(n, n)    # Generation of a random matrix
        using by default a normal distribution
    U, S, Vh = np.linalg.svd(matrix)  # Compute the svd of the matrix
    cache.append(S)                   # Saving of the Singular values

# Convertion cache to numpy
cache = np.array(cache)

# Plot of single values for all the trials
plt.figure(figsize=(12, 6))
plt.boxplot(cache.T, showfliers=False)
plt.title('Distribution of singular values')
plt.xlabel('Index')
plt.ylabel('Singular value')
ticks = [1] + list(range(10, 101, 10))
plt.xticks(ticks, [str(t) for t in ticks])
plt.savefig('Problem1.jpg')
```

Results

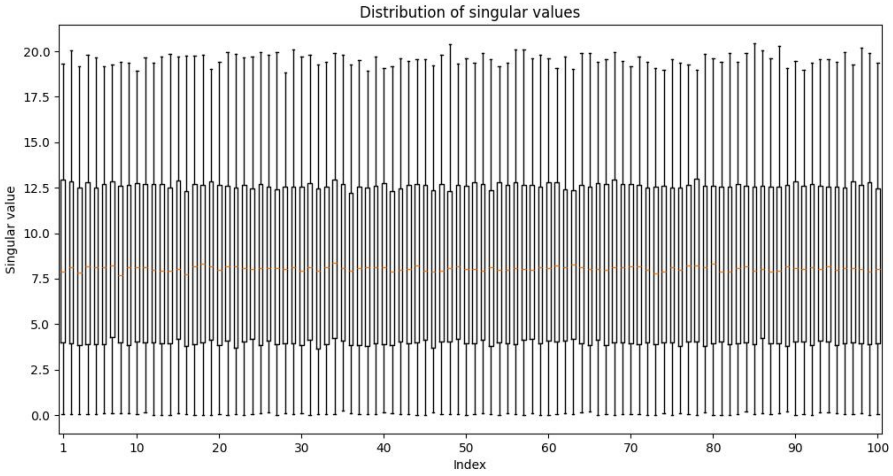

Figure 1: Distribution of singular values

Discussion

The results show that the singular values cluster around consistent magnitudes, reflecting the stability of random matrix distributions. The repetition of 100 trials emphasizes statistical convergence and shows that the distribution of singular values follows a bell-shaped tendency, resembling a normal-like pattern. This confirms that the approach was appropriate: it allowed us to capture the statistical behavior of the decomposition.

Moreover, since SVD naturally orders singular values by importance, it provides a powerful optimization framework. By discarding less significant singular values, one can approximate the original matrix with lower computational cost and reduced storage, while retaining the dominant modes that capture most of the system's structure. This illustrates why SVD is not only a useful mathematical tool but also an effective technique for dimensionality reduction and optimization in data analysis.

Problem 2: FFT Image Compression

Approach

The Fast Fourier Transform (FFT) is an efficient algorithm to compute the Discrete Fourier Transform (DFT), which expresses a signal or image in terms of its frequency components. For a two-dimensional image, the FFT decomposes the pixel intensity variations into spatial frequencies. High-magnitude coefficients correspond to dominant image structures, while low-magnitude coefficients represent finer details and noise.

This property makes FFT a valuable optimization tool: by discarding low-magnitude coefficients, one can compress the image while still preserving most of its important features. This reduces storage requirements and speeds up processing in applications such as image transmission or filtering.

In this problem, a grayscale image was transformed using the FFT, and only a percentage of the largest frequency coefficients was retained for reconstruction. The error was quantified as

$$\text{Error } (\%) = \frac{\|I - I_c\|_F}{\|I\|_F} \times 100,$$

where I is the original image and I_c the compressed version. The experiment was repeated for different compression ratios to analyze the trade-off between accuracy and efficiency.

Solution

```python
import numpy as np
from skimage import io, color
import matplotlib.pylab as plt

img = io.imread('image.jpg') # Load image
gray = color.rgb2gray(img)   # Convert it to gray scale

F = np.fft.fft2(gray)        # Compute FFT
F_shift = np.fft.fftshift(F) # Re-order of FFT

magnitude = np.abs(F_shift).ravel()   # Coumpute the magnitude
indices = np.argsort(magnitude)[::-1] # Sort index's

ratios = np.linspace(0.01, 1.0, 20) # Ratios sample
errors = []

norm_original = np.linalg.norm(gray) # Normalize gray image

for r in ratios:
    # Filter frequencies to keep
    k = int(r * len(indices))                 # n of coefficients to keep
    mask = np.zeros_like(F_shift, dtype=bool) # Create a mask
    flat_mask = mask.ravel()                  # Set all components as False
    flat_mask[indices[:k]] = True             # Mark as True the biggest coefficients
    mask = flat_mask.reshape(F_shift.shape)   # Re-shape

    F_compressed = np.where(mask, F_shift, 0) # Compressed version of FFT

    img_compressed = np.fft.ifft2(np.fft.ifftshift(F_compressed)).real # Reconstruct
        the image

    # Compute the error
    diff = gray - img_compressed
    error = (np.linalg.norm(diff) / norm_original) * 100
    errors.append(error)

# Plot of errors vs. ratios
plt.figure(figsize=(6,4))
plt.plot(ratios * 100, errors, marker='o')
plt.xlabel('Compression ratio (%)')
```

```
plt.ylabel('Error')
plt.title('FFT Compression Error vs Ratio')
plt.grid(True)
plt.savefig('Problem2.jpg')
```

Results

Figure 2: Image used.

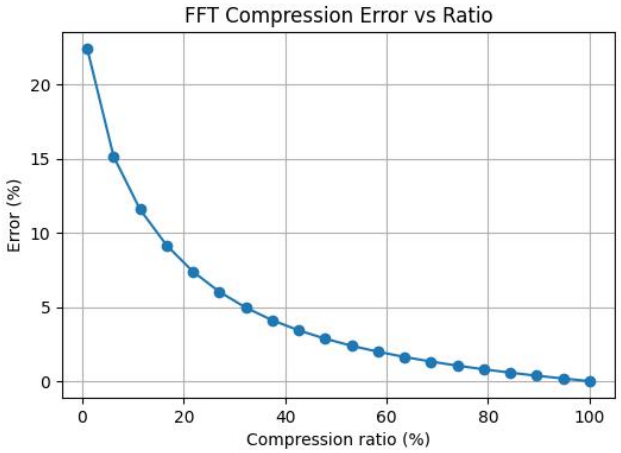

Figure 3: Compression errors in different ratios using FFT.

Discussion

The results demonstrate a clear inverse relationship between compression ratio and reconstruction error. At very low ratios (below 10%), the error is around 20%, meaning the image is still recognizable but with significant loss of detail. Between 20% and 40% of retained coefficients, the error quickly decreases to less than 5%, showing that most of the image's structure is preserved with only a fraction of the data. Beyond 60%, the error falls below 2%, and the reconstruction becomes almost indistinguishable from the original.

This validates the use of FFT as an efficient optimization technique: by prioritizing the most relevant frequency components, it achieves strong data reduction while maintaining high fidelity. Such behavior underpins practical applications in JPEG-like compression, where perceptual quality is maintained despite aggressive data reduction.